

Plano de Melhorias

GetCutt / Sistema de Barbearia

Diagnóstico: 15 de agosto de 2026

Escopo: API Node.js/Express, aplicativo Flutter, banco de dados e repositório.

Documento de referência com riscos, ações recomendadas e critérios de conclusão para preparar o projeto para produção.

Resumo executivo

O projeto possui uma base funcional, mas precisa de reforços antes da publicação. Os principais riscos são segredos versionados, autorização incompleta, preço e duração controláveis pelo cliente e ausência de proteção contra reservas simultâneas.

Ordem recomendada

1. Remover e trocar credenciais expostas.
2. Corrigir autorização e isolamento de dados.
3. Calcular preços e durações no servidor.
4. Garantir atomicidade dos agendamentos.
5. Limpar o repositório.
6. Validar entradas e proteger a API.
7. Reforçar sessões.
8. Configurar ambientes.
9. Criar testes e CI.
10. Completar a documentação.

P0 Segurança crítica

1. Remover credenciais do Git

Situação: o arquivo `.env` está versionado e contém banco e segredos JWT.

Ações:

- Adicionar `.env` ao `.gitignore` e criar `.env.example`.
- Remover segredos do índice e histórico.
- Trocar senha do banco e chaves JWT.
- Usar segredos da hospedagem e impedir inicialização com valores ausentes ou fracos.

Concluído quando: nenhum segredo real estiver no Git e todas as credenciais expostas tiverem sido rotacionadas.

2. Corrigir autorização e propriedade

Situação: algumas rotas validam o token/papel, mas não confirmam se o recurso pertence ao usuário, profissional ou barbearia.

- Cliente acessa somente seus dados e agendamentos.
- Barbeiro opera somente sua agenda e seus bloqueios.
- Administrador opera somente dentro da própria barbearia.
- Derivar identidades do token/banco e centralizar policies.
- Criar testes de acesso cruzado para todos os perfis.

Concluído quando: toda tentativa de acesso cruzado falhar com resposta apropriada.

3. Calcular preço e duração no servidor

Situação: a criação aceita `valor_override` e `duracao_override` do cliente.

- Remover esses campos da API pública.
- Receber IDs de serviços e calcular totais no backend.
- Reservar ajustes manuais para rota administrativa auditada.
- Validar limites monetários e durações.

Concluído quando: o cliente não conseguir influenciar preço ou duração com uma requisição manipulada.

P1 Integridade e confiabilidade

4. Evitar agendamento duplo

A consulta de conflito e o INSERT são separados, permitindo corrida entre requisições.

- Usar transação e bloqueio apropriado, ou slots com restrição única.
- Tratar conflito como HTTP 409.
- Testar duas requisições simultâneas.

Concluído quando: somente uma reserva concorrente for confirmada para o mesmo intervalo.

5. Usar transações em operações compostas

- Proteger criação de usuário/colaborador, agendamento/notificação e substituição de horários.
- Aplicar rollback, chaves estrangeiras, índices e restrições.
- Evitar estados parciais e registros órfãos.

6. Validar entradas e regras

- Validar schemas de body, params e query.
- Normalizar telefone/e-mail e validar IDs.
- Rejeitar datas inválidas/passadas e intervalos invertidos.
- Confirmar que serviço e colaborador estão ativos e na mesma barbearia.
- Limitar textos, valores e uploads; padronizar erros.

7. Proteger a API HTTP

- Restringir CORS por ambiente.
- Adicionar headers seguros e rate limit em autenticação.
- Validar tipo/tamanho de uploads.
- Centralizar erros e usar logs estruturados sem segredos.

8. Reforçar refresh tokens

- Armazenar somente o hash.
- Rotacionar no refresh e revogar o anterior.
- Revogar sessões de usuários desativados.
- Sincronizar o prazo persistido com a configuração JWT.
- Limpar tokens expirados e considerar armazenamento seguro no dispositivo.

P2 Organização e manutenção

9. Limpar o repositório

Foram observados cerca de 1.604 arquivos de `node_modules`, `app/lib.zip` e um repositório embutido em `backend/`, além de cópias semelhantes.

- Definir uma única fonte de verdade.
- Remover dependências e artefatos gerados do versionamento.
- Remover ou configurar corretamente o repositório embutido.
- Ampliar o `.gitignore` e manter apenas dumps sanitizados necessários.

10. Configurar ambientes no Flutter

- Substituir a URL fixa de localhost por `--dart-define` ou configuração equivalente.
- Separar desenvolvimento, homologação e produção.
- Documentar emulador Android, dispositivo físico e web.
- Exigir HTTPS fora do desenvolvimento.

11. Padronizar qualidade estática e codificação

- Padronizar UTF-8 e corrigir textos realmente corrompidos.
- Investigar o travamento observado em `flutter analyze`.
- Executar formatter/linter no CI para Flutter e JavaScript.

P2 Testes e operação

12. Criar testes automatizados

O backend não tem script de teste e o Flutter possui apenas um smoke test.

- Testar autenticação, papéis, isolamento entre barbearias e usuário desativado.
- Testar preço/duração, concorrência, cancelamento, conclusão e bloqueios.
- Testar rollback, `services/controllers` Flutter e fluxos principais.

13. Criar integração contínua

- Instalar pelos locks e executar testes, lint, análise e formatação.
- Verificar vulnerabilidades e segredos acidentais.
- Bloquear merge com verificações obrigatórias falhando.

14. Melhorar documentação

- README principal com arquitetura, requisitos e comandos.
- Variáveis de ambiente, banco/migrações e execução por plataforma.
- Papéis, regras de autorização e API em OpenAPI/Swagger.
- Health checks, backup e restauração.

15. Observabilidade e privacidade

- Logs estruturados, correlação, métricas de erro e latência.
- Health/readiness checks separados.
- Minimizar exposição de dados pessoais e estabelecer retenção alinhada à LGPD.
- Auditar operações administrativas sensíveis.

Backlog resumido

Prioridade	Item	Resultado
P0	Segredos	Credenciais expostas deixam de funcionar
P0	Autorização	Sem acesso cruzado
P0	Preço/duração	Cálculo confiável no servidor
P1	Reserva atômica	Sem agendamento duplo
P1	Validação e API	Dados consistentes e menor abuso
P1	Sessões	Tokens rotativos e revogáveis
P2	Estrutura e ambientes	Fonte única e configuração reproduzível
P2	Testes, CI e docs	Manutenção previsível

Estado observado em 15/08/2026. Antes das mudanças, preservar o trabalho local e criar backup do banco. Retirar o .env do commit atual não remove segredos do histórico; a rotação é obrigatória.